

RESEARCH

Real-Time Detection Algorithm of Port Ships Based on Improved YOLOv11

Boweng Xing¹ · Qingsong Xing¹ · Chengwu Pan² · Qibo Le² · Fujia Bi³

Received: 20 March 2025 / Revised: 26 September 2025 / Accepted: 21 October 2025

© The Author(s) 2025

Abstract

To address the challenge of balancing accuracy and real-time performance in ship detection for ports and waterways, this paper proposes an optimized detection algorithm named YOLOv11-FFCM, based on an enhanced YOLOv11 framework. The proposed method integrates the Focal Modulation module, FasterNetBlock structure, and CBAM attention mechanism to strengthen feature extraction and improve detection performance in scenarios with complex backgrounds, small targets, and low visibility. The experimental results show that the YOLOv11-FFCM achieves superior detection accuracy while maintaining an effective trade-off between computational complexity and real-time performance. These advantages make it highly suitable for practical applications in port and waterway ship surveillance. Compared with YOLOv11, YOLOv11-FFCM improves 4.4% of mAP@0.5 and 4.4% of mAP@0.5:0.95 on the HarbourShip dataset and 2.2% of mAP@0.5 and 2.4% of mAP@0.5:0.95 on the Unreal-Vessels-detection-v1 dataset.

Keywords Port ship target detection · YOLOv11 · Real-time object detection · FocalModulation · FasterNet · CBAM

1 Introduction

Ship detection plays a pivotal role in global port and waterway management, serving as a critical technology for ensuring navigational safety, optimizing port scheduling, and enhancing logistics efficiency [1]. Traditional ship monitoring methods, such as radar and the Automatic Identification System (AIS), have been established for maritime surveillance for a long time. However, these systems often face limitations in target resolution and real-time responsiveness [2]. Over the years, research has evolved from traditional image processing techniques to advanced deep learning methods, aiming to address the challenges posed by dynamic and cluttered port environments.

In the early stages of ship detection research, much of the focus was on context modeling and motion analysis to support real-time surveillance. Notably, Bao et al. [3] introduced a practical framework that successfully combined these techniques, demonstrating its suitability for embedded systems operating with non-static cameras and inherently challenging setting for many traditional algorithms. Zhi et al. [4], for instance, significantly enhanced detection performance by refining sea clutter modeling, which involved careful removal of land pixels and false targets.

With the rapid development of artificial intelligence, deep learning has demonstrated considerable potential in the domain of ship detection [5] [6]. Compared to traditional approaches that rely on handcrafted features, deep learning models exhibit superior abilities in feature extraction and multi-scale object perception, thereby significantly

Boweng Xing and Qingsong Xing have contributed equally to this work



improving detection accuracy and adaptability [7]. Object detection frameworks are generally categorized into two-stage detectors (e.g., Faster R-CNN [8]) and one-stage detectors (e.g., YOLO [9], SSD [10]), with the YOLO series being particularly popular for real-time applications due to its simple architecture, fast inference speed, and suitability for edge deployment [11]. Zhao et al. [12] introduced an embedded system (ESDR-DL) that demonstrated robust, long-term performance in real-world port settings. Deep models have also been incorporated into risk assessment processes; Liu et al. [13] utilized autoencoder-based anomaly detection to clean historical motion data, enabling accurate estimation of collision frequencies and identification of high-risk areas.

Optical remote-sensing images present unique challenges as ships often share similar textures and grayscale tones with surrounding infrastructure. To address this, Wenbin et al. [14] combined visual saliency, dark channel priors, and watershed segmentation techniques to effectively separate ships from background clutter. Similarly, Hu et al. [15] developed a lightweight multi-scale CNN tailored for nearshore detection in high-resolution imagery, achieving a detection rate of 96.92% with minimal false alarms.

Advancements in trajectory analysis have significantly enhanced collision detection and risk assessment. Shi et al. [16] proposed an algorithm that accounts for variable time intervals between trajectory pairs, improving the detection of both maximum and average speed differences' key factors in evaluating collision risks.

Recent efforts have also focused on integrating machine learning models for ship detection and classification. Kim et al. [17] applied the YOLO algorithm to identify container ships in CAS500-1 imagery, specifically targeting those beyond the traditional AIS coverage, thereby addressing the limitations of AIS-based systems.

The application of deep learning in smart ports and smart cities has further advanced maritime surveillance capabilities. Bakirci et al. [18] employed the Faster R-CNN model to detect ships in coastal and harbor environments, demonstrating its potential to enhance both maritime security and urban resilience.

In addition, the detection of anomalous ship behaviors has gained attention for improving waterway traffic monitoring. Li et al. [19] proposed methods based on text similarity and kernel density estimation to identify abnormal behaviors, such as sudden acceleration or deceleration, contributing to more proactive traffic management strategies.

To further enhance the accuracy and efficiency of real-time ship detection, researchers have explored various optimization strategies. Some approaches integrate attention mechanisms and feature fusion to improve performance under challenging conditions. For instance, Qiu et al. [20] employed Adaptive Spatial Feature Fusion (ASFF) to improve multi-scale detection accuracy. Misra et al. [21] introduced Transformer-based architectures with window attention to enhance robustness in complex scenarios. Jin et al. [22] applied the SENet attention mechanism to pyramid pooling for adaptive channel-wise recalibration, thereby strengthening feature representation.

In summary, the field of port ship detection encompasses a broad range of methodologies, including motion analysis, image segmentation, deep learning-based classification, and behavior anomaly detection. These diverse approaches collectively aim to improve the accuracy, reliability, and real-time performance of port surveillance systems, thereby strengthening both maritime safety and operational efficiency.

Despite these advancements, a key challenge in port ship detection remains: achieving an optimal trade-off between detection accuracy and real-time performance, particularly in scenarios with complex backgrounds, occlusion, low visibility, and small targets at long distances.

To address this challenge, this paper proposes a novel detection algorithm, YOLOv11-FFCM, which is based on an enhanced YOLOv11 framework. The proposed method incorporates the Focal Modulation module, which refines features through hierarchical contextualization, gated aggregation, and affine transformation. Additionally, the FasterNetBlock structure replaces the Bottleneck modules in the C3k2 component of YOLOv11, thereby forming a more efficient feature extraction network. Moreover, the CBAM attention mechanism is integrated to strengthen both channel and spatial attention, without increasing the channel dimensionality. This ensures that the most discriminative features are emphasized, enhancing the model's performance under adverse conditions.

The main contributions of this paper are summarized as follows:

- A novel real-time ship detection algorithm, YOLOv11-FFCM, is proposed to improve accuracy and robustness in challenging maritime environments.
- The integration of Focal Modulation, FasterNetBlock, and CBAM significantly enhances feature representation by refining the model's ability to focus on the most critical and discriminative features. This combination effectively captures complex spatial relationships and channel-wise dependencies, all while maintaining low computational complexity, ensuring efficient performance in real-time ship detection tasks.
- In this paper, ablation studies and comparative performance evaluations are conducted to comprehensively assess the effectiveness of the proposed model for ship detection under challenging conditions, including complex backgrounds, occlusion, low visibility, and small targets at long distances. Experimental results demonstrate that the proposed method delivers superior performance, achieving 94.8% mAP@0.5 and 78.1% mAP@0.5:0.95 on the HarbourShip dataset, as well as 98.8% mAP@0.5 and 85.6% mAP@0.5:0.95 on the Unreal-Vessels-detection-v1 dataset.

The main contents of this paper are as follows: Sect. 2 introduces the YOLOv11-CCFM proposed in this paper and its innovations. Section 3 introduces the relevant experimental preparations, analyzes the experimental results, and verifies the detection performance of the improved algorithm proposed in this paper. Section 4 gives the main conclusions and the direction for further improvement of the algorithm in the future.

2 Principle and Improvements

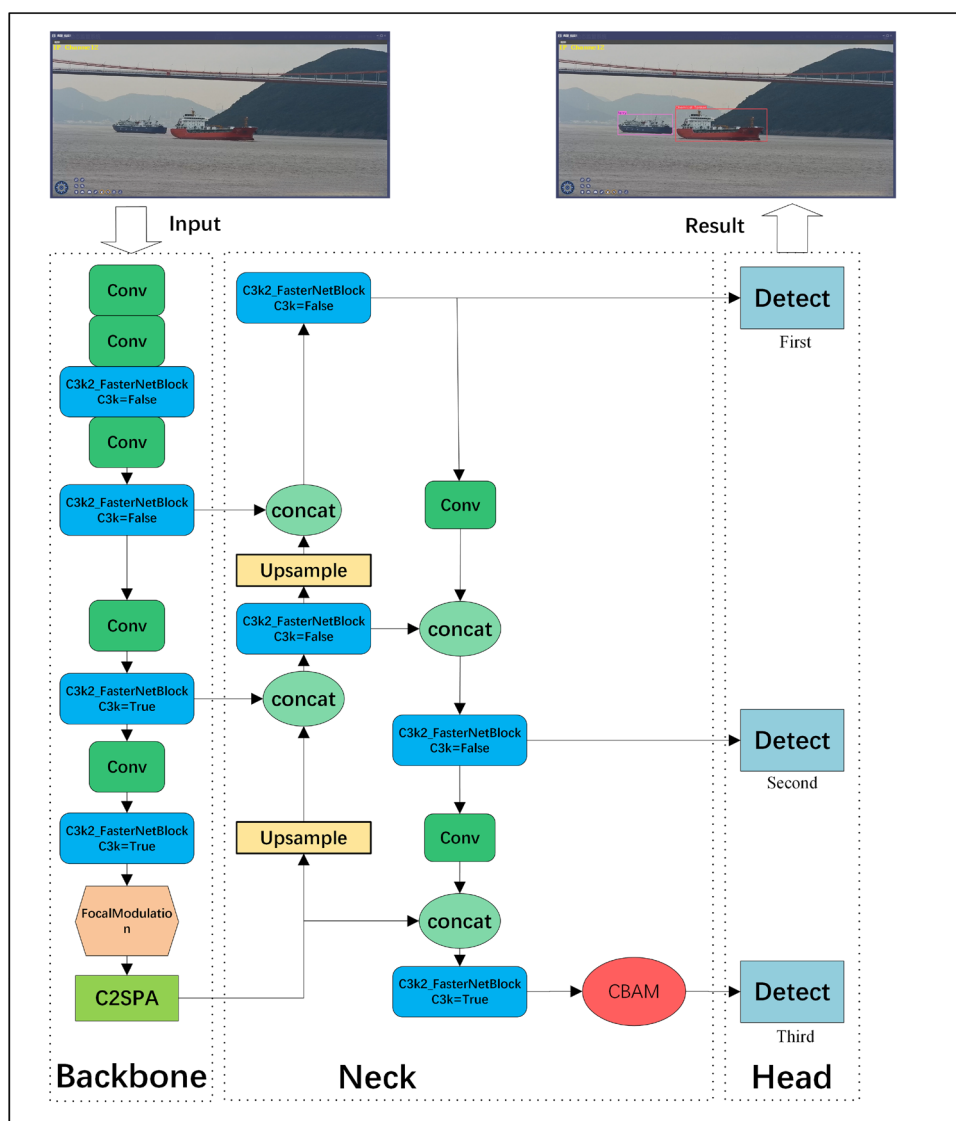
2.1 YOLOv11-FFCM

In this study, to improve the performance of YOLOv11 in the real-time detection task of port ships, we made several improvements to the original network, mainly including introducing the FocalModulation module, FasterNetBlock, and CBAM attention mechanism. We named the improved algorithm YOLOv11-FFCM. The core goal of these improvements is to achieve the best balance between detection accuracy, inference speed, and robustness. The overall structure of the YOLOv11-CCFM in Fig. 1.

First, this paper introduces the Focal Modulation module to replace the original SPFF component. While the SPFF module is effective for multi-scale feature extraction, it often suffers from information loss and computational redundancy in complex scenarios, particularly for detecting small or distant targets. By aggregating local contextual information and interacting with query features, the Focal Modulation module effectively captures both long-range dependencies and fine-grained details, thereby enhancing detection accuracy. Moreover, its low computational complexity ensures improved accuracy without substantially increasing the overall computational burden. With this modification, the network demonstrates stronger capability in detecting ship targets within complex port environments, especially small-scale targets.

Second, this study incorporates the FasterNetBlock to enhance the C3k2 module. As a critical component for feature extraction in YOLOv11, the performance of C3k2 directly influences the overall detection effectiveness. To further improve feature extraction efficiency, FasterNetBlock replaces the original Bottleneck structure. Leveraging partial convolution, FasterNetBlock applies convolution operations to only a subset of the input feature map, reducing redundant computation and memory access. Subsequently, pointwise convolution is employed for feature integration, with normalization and activation layers introduced to preserve feature diversity and integrity. This design significantly reduces computational cost and inference latency while strengthening feature expressiveness, enabling efficient performance in high-resolution, multi-object detection tasks.

Finally, the CBAM attention mechanism is integrated into the detection head to enhance detection accuracy and improve robustness against background interference. In complex port environments, multi-target scenes and cluttered backgrounds often lead to false alarms or missed detections. CBAM addresses this issue by sequentially applying channel and spatial attention modules, adaptively weighting features to emphasize the most discriminative

Fig. 1 YOLOv11-CCFM structure

information. Specifically, the channel attention module evaluates the importance of each feature channel via global pooling and a shared MLP, while the spatial attention module aggregates spatial cues using maximum and average pooling operations to highlight target regions. This integration enables the network to more effectively distinguish ships from background noise, substantially reducing false and missed detections.

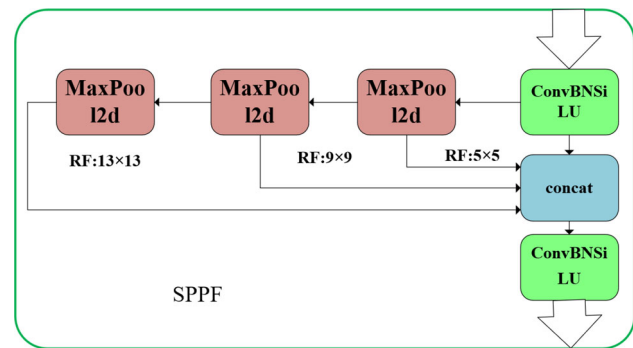
Through these enhancements, YOLOv11-FFCM achieves an improved balance of accuracy, inference speed, and robustness, offering strong technical support for practical deployment in port ship detection tasks.

2.2 High-Precision Spatial Pyramid Pooling Module: FocalModulation

2.2.1 Spatial Pyramid Pooling

Spatial Pyramid Pooling (SPP) is a network layer specifically designed to address the fixed input size limitation of convolutional neural networks (CNNs). In standard CNN architectures, input images typically need to be resized to a uniform dimension to satisfy the requirements of fully connected layers. However, this resizing can lead to information loss or image distortion, which may negatively impact the model's detection and classification performance. SPP overcomes this limitation by performing pooling operations at multiple scales, allowing the extraction

Fig. 2 SPPF Model Structure



of features from different receptive fields. As a result, the network can process input images of arbitrary sizes while generating fixed-length output feature vectors. This capability is particularly valuable in image classification and object detection tasks, as it enhances both the model's generalization ability and its detection performance.

Spatial Pyramid Pooling Fast (SPPF) is a technology widely used in deep learning and computer vision, especially in object detection tasks. It is an efficient variant optimized based on Spatial Pyramid Pooling (SPP), which aims to improve computational efficiency while maintaining multi-scale feature extraction capabilities [26]. The structure of SPPF is shown in Fig. 2.

2.2.2 FocalModulation

The algorithm proposed in this paper is mainly designed for waterway safety monitoring scenarios, with the goal of deployment on resource-constrained edge computing devices. Since edge devices face significant limitations in terms of memory, storage, and computational power, this paper replaces the original self-attention mechanism in the YOLOv11 model with the focal modulation mechanism to substantially improve inference efficiency.

Compared with the self-attention mechanism, whose computational complexity grows quadratically with the sequence length n ($O(n^2)$), the focal modulation mechanism leverages depth-wise convolution and hierarchical context aggregation strategies to reduce computational complexity to a linear level ($O(n)$). This enables superior efficiency when processing long sequence data such as images and video streams. The mechanism significantly reduces memory consumption and enhances the compatibility and feasibility of model deployment on edge hardware.

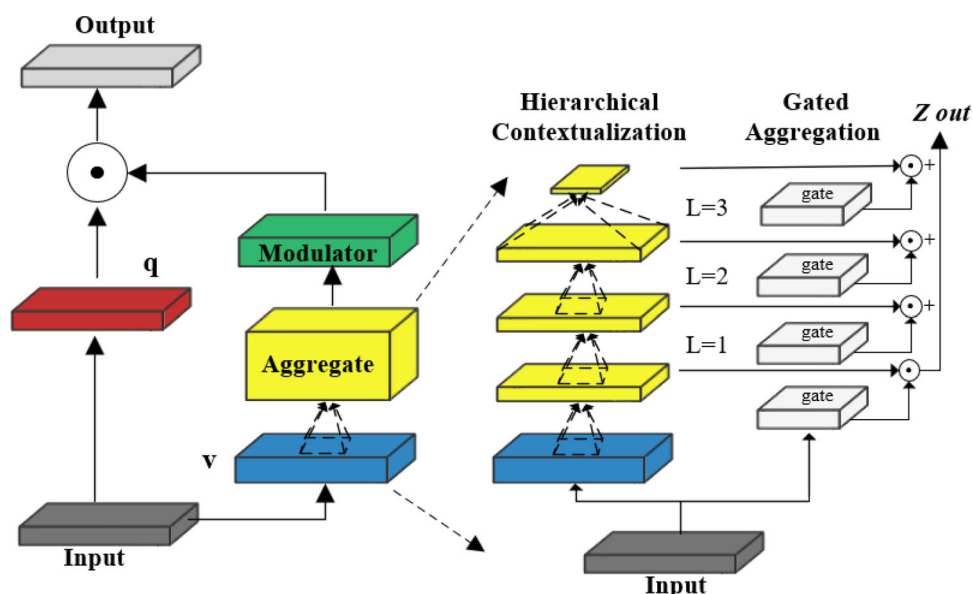
The Focal Modulation mechanism is a novel feature enhancement approach that effectively captures long-range dependencies and contextual information in images [27]. It comprises three key components: Hierarchical Contextualization, Gated Aggregation, and Element-wise Affine Transformation. The overall structure of the module is illustrated in Fig. 3.

Hierarchical Contextualization encodes visual information across different spatial ranges using a series of depth-wise convolutional layers, effectively capturing features from near to far. This approach enables the network to represent both local and global features, allowing it to understand image content at multiple scales while simultaneously enhancing the perception of fine details and overall structural context.

The process follows: First, the input feature map X is projected to the new space $Z_0 = f_z(X)$ through a linear transformation. Then, L layers of deep convolution are gradually used to extract local to global multi-scale features

$$\begin{aligned} Z &= f_a(Z_{l-1}) \\ &= \text{GeLU}(\text{DWConv}(Z_{l-1})). \end{aligned} \quad (1)$$

Here, Z_{l-1} represents the feature map from the $(L - 1)$ th layer. After processing with depth-wise convolution (DWConv), a new feature map Z is generated and passed through a GeLU activation for nonlinear transformation. The features of the L th layer are then subjected to global average pooling to produce the $(L + 1)$ th layer feature map, effectively capturing contextual information across multiple scales.

Fig. 3 Focal modulation structure

Gating mechanisms are commonly employed in deep learning to regulate the flow of information, determining which signals should be propagated and which should be suppressed. In the Focal Modulation process, the Gated Aggregation mechanism plays a central role. Its core principle is to filter contextual information through the gating operation, retaining only the most informative context features for the current query token. This selective process enhances the representational power and expressiveness of the model.

Gated aggregation mainly includes the following steps: (1) using deep convolutional layers to extract contextual information of different scales; (2) adaptively selecting information that is valuable to the query token through the gating mechanism; (3) aggregating the contextual information into the modulator according to the gating weight for subsequent feature modulation. Its calculation method is as follows:

$$Z_{\text{out}} = \sum_{l=1}^{L+1} G_l \odot Z_l; \quad (2)$$

Z_l represents the multi-scale features obtained from different layers. G_l is the gating weight calculated by a linear transformation, which indicates the importance of the features of each layer to the final representation. $\odot$ represents the element-by-element multiplication, which applies the weight of each feature layer to the feature map. The final output Z_{out} is the weighted sum of all layer features, realizing multi-scale information fusion.

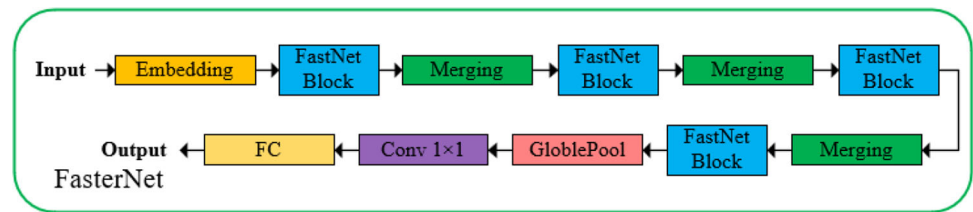
The advantage of gated aggregation is that it can effectively filter out redundant information and retain only the feature information that contributes to the target detection task, thereby improving the model's computational efficiency and detection performance.

The element-by-element radiometric transformation is the last step of the Focal Modulation mechanism, which aims to inject the modulator information after gated aggregation into the query token to enhance the feature expression. This transformation adopts the element-by-element linear transformation form. Through the linear transformation, the contextual information in the modulator can directly affect the feature expression of the query token. Its calculation method is as follows:

$$y = \gamma \odot x + \beta. \quad (3)$$

Among them, x is the input feature, y is the transformed output, γ and β are learnable affine parameters, representing scaling and shifting, and $\odot$ represents element-wise multiplication.

Fig. 4 FasterNet structure



Combining the above, the calculation method of the Focal Modulation mechanism is as follows:

$$y_i = q(x_i) \odot h \left(\sum_{l=1}^{L+1} g_i^l z_i^l \right); \quad (4)$$

$q(x_i)$ is the linear projection of the query feature, which is used to extract key information, g_i^l is the gating weight of the L th layer, corresponding to the value of G at space i , z_i^l is the value of the L th layer feature at position i , and $h()$ is the element-by-element affine transformation.

2.3 High-Efficiency Feature Extraction Network: C3k2-FasterNetBlock

2.3.1 FasterNet

FasterNet is an efficient neural network architecture primarily composed of Partial Convolution (PConv) and acceleration-oriented network designs. It is specifically developed to substantially enhance computational speed without compromising accuracy, making it particularly well suited for visual tasks [28].

Partial Convolution (PConv) is an efficient operation designed to improve computational performance. Unlike traditional convolution, PConv applies convolution only to a subset of the input feature map rather than the entire map. Skipping redundant or irrelevant regions reduces unnecessary computation and memory access, thereby lowering computational cost. This approach is especially advantageous for resource-constrained devices, enabling deep learning models to operate more efficiently while maintaining high performance.

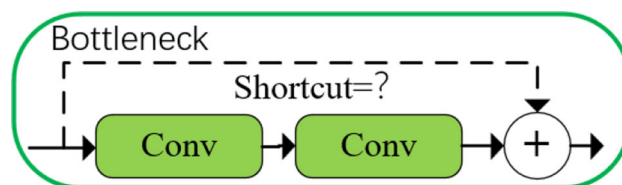
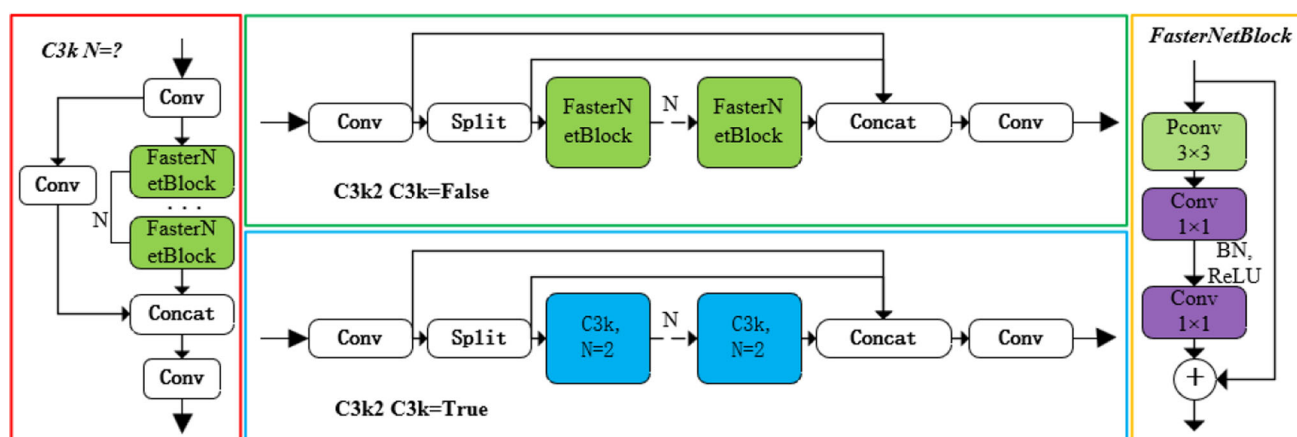
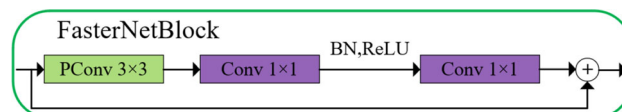
Accelerated neural network techniques further reduce inference time by optimizing computational paths, simplifying network structures, and leveraging hardware-efficient operations. These strategies may include streamlining network layers, using faster activation functions, employing quantization to convert floating-point operations into integer operations, and designing memory-efficient algorithms. Collectively, these optimizations allow neural networks to process data and generate predictions more rapidly, ensuring accuracy while improving operational efficiency in practical applications.

The overall structure of FasterNet is illustrated in Fig. 4. It comprises four hierarchical stages, each consisting of a sequence of FasterNetBlocks and beginning with an embedding or merging layer. The final three layers are dedicated to feature classification.

2.3.2 C3k2-FasterNetBlock

In this paper, the feature extraction network is optimized by replacing the Bottleneck module in the C3k2 component with FasterNetBlock from FasterNet, thereby constructing a more efficient feature extraction network.

The Bottleneck structure is a module that optimizes the computational efficiency of convolutional neural networks (CNNs) and is widely used in ResNet and other efficient feature extraction networks [29]. Its core idea is to reduce the amount of computation by reducing and increasing the dimension while maintaining the expressive power of the model, thereby improving the computational efficiency of the network.

Fig. 5 Bottleneck structure**Fig. 6** FasterNetBlock structure**Fig. 7** C3k2-FasterNetBlock Structure

The Bottleneck structure consists of three layers of convolution. First, the 1×1 convolution reduces the dimension and number of channels, reducing the subsequent computational cost. Next, the 3×3 convolution performs local feature extraction on the feature map after dimension reduction to retain important information. Finally, the 1×1 convolution increases the dimension and restores the original number of channels to ensure that information is not lost. The Bottleneck structure is usually combined with a shortcut connection. The input data can skip the three convolution layers directly and be added to the output element by element to alleviate the gradient vanishing problem and accelerate network convergence. Its structure is shown in Fig. 5.

The FasterNetBlock serves as the core computational unit of the FasterNet architecture. Its primary characteristic is the use of Partial Convolution (PConv) for efficient computation, combined with pointwise convolutions to optimize information flow [30]. Within the FasterNetBlock, the PConv layer is followed by two pointwise convolution layers to further enhance feature representation and computational efficiency. To preserve feature diversity while minimizing inference latency, Batch Normalization and the ReLU activation function are applied only between the two pointwise convolution layers, rather than after each convolution. This design reduces redundant computations and accelerates model convergence, ensuring that rich feature representations are maintained even in deep networks while optimizing the use of computational resources. As a result, FasterNet can sustain efficient performance across various application scenarios. The structure of the FasterNetBlock is illustrated in Fig. 6.

Thanks to the PConv technology, computations are performed only on a subset of the input feature map, thereby reducing memory access requirements and overall computational cost. The enhanced C3k2-FasterNetBlock feature extraction network achieves higher computational speed without sacrificing accuracy, improving the model's operational efficiency. This makes the network particularly well suited for resource-constrained environments, such as edge computing devices and mobile applications. The structure of the C3k2-FasterNetBlock module is illustrated in Fig. 7.

Fig. 8 CBAM structure

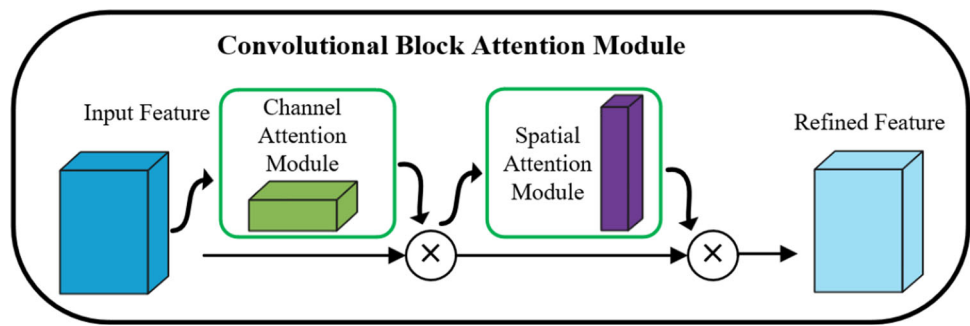
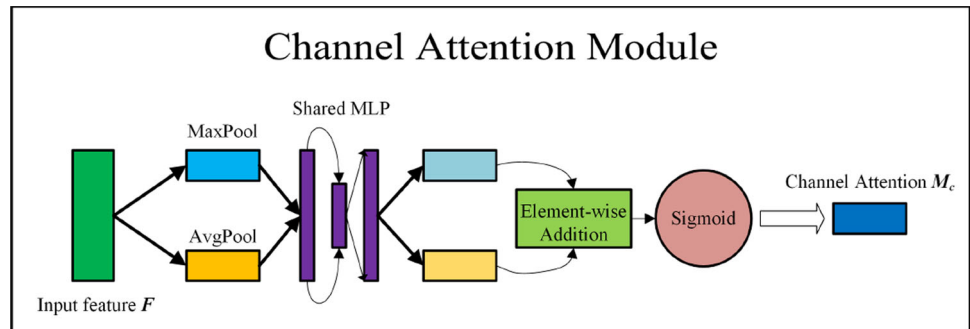


Fig. 9 CAM structure



2.4 Lightweight Spatial Channel Attention Module: CBAM

Convolutional Block Attention Module (CBAM) is a lightweight and efficient attention mechanism designed to enhance the feature representation capability of CNNs by explicitly modeling relationships across both channel and spatial dimensions [31]. CBAM consists of two sequential sub-modules: the Channel Attention Module (CAM) and the Spatial Attention Module (SAM). First, channel attention is applied to emphasize informative feature channels, followed by spatial attention to focus on important spatial regions, enabling the model to capture key information along both dimensions.

Specifically, given an input feature map $\mathbf{F}$, the channel attention map $\mathbf{M}_c(\mathbf{F})$ is computed through CAM and applied to $\mathbf{F}$ via element-wise multiplication ($\otimes$), producing the weighted feature map $\mathbf{F}'$. Next, $\mathbf{F}'$ is processed by SAM to generate the spatial attention map $\mathbf{M}_s(\mathbf{F}')$, which is again applied element-wise to $\mathbf{F}'$ to yield the optimized output feature $\mathbf{F}''$. The overall structure of CBAM is illustrated in Fig. 8.

The calculation method is as follows:

$$\mathbf{F}' = \mathbf{M}_c(\mathbf{F}) \otimes \mathbf{F} \quad (5)$$

$$\mathbf{F}'' = \mathbf{M}_s(\mathbf{F}') \otimes \mathbf{F}'. \quad (6)$$

The CBAM structure can be directly embedded into standard CNN structures (such as ResNet, MobileNet, and EfficientNet) without introducing too much computational overhead and, therefore, has been widely used in various computer vision tasks (such as image classification, object detection, and image segmentation).

2.4.1 CAM

The core principle of the Channel Attention Module (CAM) is to preserve the channel dimension while compressing the spatial dimension, thereby emphasizing the most discriminative information in the input feature map. For instance, in a classification task, CAM enables the model to focus on the features that are most influential in distinguishing between different categories. The structure of the module is illustrated in Fig. 9.

The input feature map is first processed by global average pooling and max pooling, reducing it from $C \times H \times W$ to $C \times 1 \times 1$. These pooled features pass through a shared MLP, which reduces and then restores the channel dimension, with ReLU applied in between. The resulting outputs are combined and passed through a Sigmoid to produce the channel attention weights. Multiplying these weights with the original feature map enhances important channels while suppressing irrelevant ones. Although the output retains the original spatial dimensions, its representation along the channel dimension is optimized, improving the network's focus on key features.

The channel attention weight calculation method is as follows:

$$\begin{aligned} M_c(\mathbf{F}) &= \sigma (MLP(AvgPool(\mathbf{F})) + MLP(MaxPool(\mathbf{F}))) \\ &= \sigma \left(\mathbf{W}_1 \left(\mathbf{W}_0(\mathbf{F}_{avg}^c) \right) + \mathbf{W}_1 \left(\mathbf{W}_0(\mathbf{F}_{max}^c) \right) \right) \end{aligned} \quad (7)$$

$$F_{avg}(c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X(c, i, j) \quad (8)$$

$$F_{max}(c) = \max_{i \in [1, H], j \in [1, W]} X(c, i, j). \quad (9)$$

The input feature map size is $F \in R^{C \times H \times W}$, and $M_c(\mathbf{F})$ represents the channel attention map. σ represents the Sigmoid activation function. MLP represents the multi-layer perceptron mechanism. $\mathbf{F}_{avg}^c$ and $\mathbf{F}_{max}^c$ represent the features after channel-level average pooling and maximum pooling. $\mathbf{W}_0$ and $\mathbf{W}_1$ represent the two fully connected layers in the MLP ; $\mathbf{W}_0 \in R^{\frac{C}{r} \times C}$ is the dimension reduction matrix, r is the reduction rate, and $\mathbf{W}_1 \in R^{C \times \frac{C}{r}}$ is the dimension increase matrix.

2.4.2 SAM

The Spatial Attention Module (SAM) is an attention mechanism in deep neural networks that focuses on enhancing the model's attention to key spatial regions, thereby improving the overall performance of visual tasks. Unlike the Channel Attention Module (CAM), the core goal of SAM is to learn the importance of feature maps in the spatial dimension rather than the information in the channel dimension. It helps the model select features in the $H \times W$ dimension (i.e., spatial position) rather than distinguishing the importance of information in the channel dimension.

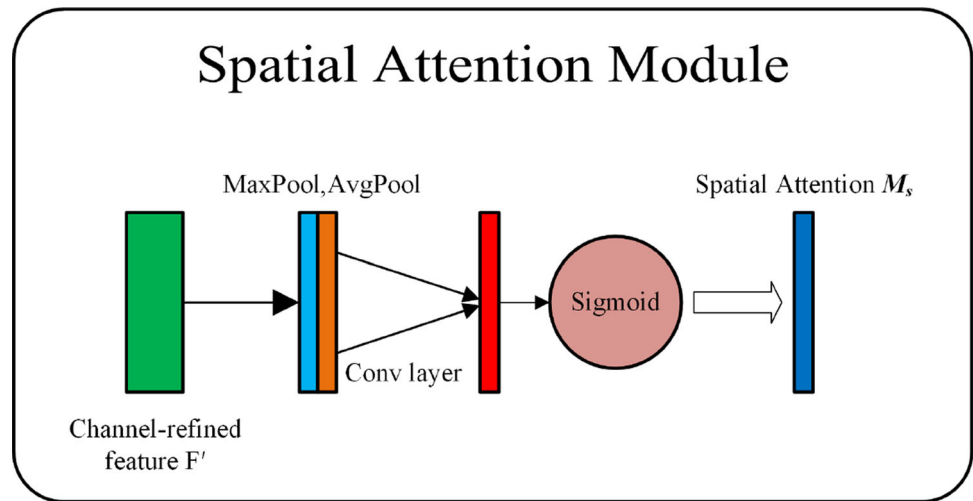
SAM automatically identifies key pixels that contribute to the task by calculating the global spatial attention weights of the feature map while suppressing information in irrelevant areas, thereby improving the model's perception ability. This process enables the model to focus more accurately on the key areas of the target object, improve the model's ability to understand complex scenes, reduce unnecessary waste of computing resources, and improve overall reasoning efficiency. This feature of SAM is particularly prominent in tasks, such as object detection, semantic segmentation, image classification, and image generation. It can effectively enhance the model's recognition ability and generalization performance.

To achieve spatial attention, SAM uses a combination of max pooling and average pooling to extract global and local spatial information from the feature map. Then, the two pooling features are spliced in the channel dimension and fused through a 7×7 convolutional layer to capture local spatial relationships. Then, the Sigmoid activation function generates the spatial attention weights after normalization. Finally, SAM applies the attention weights to the original feature map by element-by-element weighting, allowing the model to focus more on the features of key areas while suppressing irrelevant information, thereby improving the effectiveness of the task. The SAM structure is shown in Fig. 10.

The spatial attention weight is calculated as follows:

$$M_s(\mathbf{F}) = \sigma \left(f^{7 \times 7} ([AvgPool(\mathbf{F}); MaxPool(\mathbf{F})]) \right)$$

Fig. 10 SAM structure



$$= \sigma \left(f^{7 \times 7} \left([\mathbf{F}_{avg}^s; \mathbf{F}_{max}^s] \right) \right); \quad (10)$$

$M_s(\mathbf{F})$ represents the spatial attention weight matrix, σ represents the Sigmoid activation function, which is used to normalize the output, $f^{7 \times 7}$ represents the 7×7 convolution kernel, which is used to extract spatial features, and $\mathbf{F}_{avg}^s$ and $\mathbf{F}_{max}^s$ represent the feature maps after pooling.

3 Experiment and Discussion

3.1 Dataset

The dataset used to train the object detection algorithm proposed in this paper is derived from real-world ship surveillance footage captured by monitoring cameras in the Zhoushan Port waterways. The images were carefully selected to include data under extreme conditions, such as foggy weather, low illumination, complex backgrounds, and occlusion. A total of 2400 images were collected, encompassing six major categories: tug boats (400 images), cargo ships (400 images), chemical tankers (400 images), ferries (400 images), bulk carriers (400 images), and small boats (400 images). This dataset is named HarbourShip.

The HarbourShipdataset exhibits the following key characteristics:

1. **Complex backgrounds:** Images feature challenging visual scenes, including nearshore port infrastructure, mountainous terrain, and bridges.
2. **Significant variation in ship sizes:** Large differences in ship scales make small target detection particularly difficult when both large and small vessels appear simultaneously.
3. **Diverse ship appearances and colors:** For example, chemical tankers are predominantly red, tugboats are typically white, and ferries are mostly blue.
4. **Intra-class variation:** Ships within the same category, especially cargo ships, exhibit substantial variability in structure and appearance.

Figure 11 shows several representative samples from the HarbourShip dataset.

After image acquisition, the LabelImg tool was used to manually annotate the ships in the dataset. The annotated dataset was then divided into training, validation, and test sets in a ratio of 7:1:2.

Fig. 11 Dataset example

3.2 Performance Evaluation

To verify the effectiveness of the improved YOLOv11-FFCM model, this paper conducts comparative experiments with the original YOLOv11 model, the original YOLOv5n model, the original YOLOv8n model, SSD, RT-DETR, and YOLOX algorithms. To objectively evaluate the performance of each model, the following metrics are introduced: *Precision*, *Recall*, $mAP@0.5$, $mAP@0.5 : 0.95$, *FLOPs*, and *FPS*.

Precision indicates how many of the samples predicted as positive are correct and is calculated as follows:

$$Precision = \frac{TP}{TP + FP}; \quad (11)$$

TP means that both the predicted and actual results are positive, and FP means that the predicted result is positive, but the actual result is negative.

Recall indicates how many true positive samples are predicted correctly and is calculated as follows:

$$Recall = \frac{TP}{TP + FN}; \quad (12)$$

FN means the predicted result is negative, but the true result is positive.

The mAP refers to the average of all categories of average precision, which is used to measure the detection performance of the model for all ship types. It is calculated as follows:

$$AP = \int_0^1 Precision(Recall) d(Recall) \quad (13)$$

Table 1 Training hyperparameters

Hyperparameters	Value
Epoch	300
Batch Size	16
Image Size	640×640
Optimizer	SGD
Momentum	0.937
Patience	50
Workers	8
Learning Rate	0.001
Weight Decay	0.0005
Warmup Epochs	3
Warmup Momentum	0.8
Warmup Bias Learning Rate	0.1
Pretrained	False
Mosaic	True
MixUp	False

$$mAP = \frac{1}{n} \sum_{k=1}^{k=n} AP_K; \quad (14)$$

AP_K refers to the average accuracy of category K , and n refers to the number of categories. $mAP@0.5$ represents the mean Average Precision of the model in detecting all ship categories when the Intersection over Union (IoU) threshold is set to 0.5. $mAP@0.5 : 0.95$ represents the mean Average Precision computed across multiple IoU thresholds ranging from 0.5 to 0.95 in steps of 0.05. Compared with $mAP@0.5$, $mAP@0.5 : 0.95$ is a more stringent and comprehensive evaluation metric, as it assesses the model's detection performance under varying degrees of localization precision.

$FLOPs$ (Floating Point Operations) is an important metric used to evaluate the computational complexity of a model and its demand on hardware resources. A lower FLOPs value generally indicates a more lightweight model that is more suitable for deployment on devices with limited computational power. FPS (Frames Per Second), on the other hand, measures the real-time detection capability of the model. A higher FPS value implies faster inference speed, which is crucial for applications requiring real-time performance, such as maritime surveillance and port traffic monitoring.

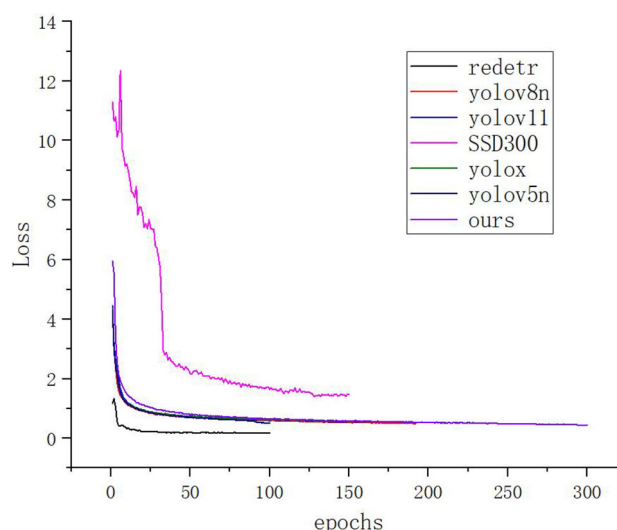
3.3 Environment and Configuration

The experiments in this paper were conducted on a system equipped with an AMD 5600X processor (6 cores, 12 threads, 3.7 GHz–4.6 GHz), 16 GB of RAM, and an NVIDIA RTX 2070 GPU with 8 GB of video memory. The operating system used was Windows 10. The deep learning framework was PyTorch 2.3.1, with Python version 3.10 and CUDA version 11.8. The improved YOLOv11 model was trained from scratch without using pre-trained weights. The specific training hyperparameters are shown in Table 1.

3.4 Comparison Experiments

3.4.1 Comparison with Mainstream Detection Algorithms

To evaluate the effectiveness of the proposed algorithm, comparative experiments were conducted against several mainstream detection methods, including YOLOv5n, YOLOv8n, YOLOX, SSD, and RT-DETR, using the Har-

Fig. 12 Training loss curve**Table 2** Performance of different detection methods on the HarbourShip dataset

Method	P	R	mAP@0.5	mAP@0.5:0.95	FLOPs/G	FPS/s ⁻¹
YOLOv5n	84.5%	84.2%	93.4%	73.8%	7.1	96
YOLOv8n [32]	93.0%	81.7%	91.0%	75.3%	8.1	132
YOLOv11	90.6%	80.0%	90.4%	73.7%	6.3	97
RTDETR-L [33]	93.3%	87.8%	93.6%	77.8%	103.5	24
SSD300 [10]	92.6%	85.7%	94.3%	76.2%	61.3	35
YOLOX-s [34]	90.5%	84.4%	94.2%	77.5%	26.8	50
Ours	93.5%	87.8%	94.8%	78.1%	7.5	90

bourShip dataset. The training loss curve is presented in Fig. 12, and the experimental results are summarized in Table 2.

After analyzing the experimental results, the improved algorithm demonstrates significant improvements over the original YOLOv11 across several key metrics on the HarbourShip dataset. Specifically, the precision increased by 2.9%, recall improved by 7.8%, mAP@0.5 rose by 4.4%, and mAP@0.5:0.95 improved by 4.4%. Additionally, compared to YOLOv8, RT-DETR-L, SSD300, and YOLOX-s, this algorithm showed improvements in detection accuracy, indicating its superior performance in object detection tasks.

The algorithm is primarily designed for real-time ship detection in the Zhoushan port channel, which requires not only high detection accuracy but also needs to balance computational complexity and real-time performance. From the experimental results, although YOLOv8n outperforms the proposed algorithm in terms of frame rate (FPS), its detection accuracy is significantly lower, and its computational complexity (FLOPs) is higher, indicating greater dependency on hardware resources and a weaker ability to balance detection accuracy and real-time performance. At the same time, algorithms like RT-DETR-L, SSD300, and YOLOX-s, although showing some advantage in detection accuracy, are still slightly inferior to the proposed algorithm. Their FPS values are 24, 35, and 50, respectively, which are lower than the frame rate of the port channel surveillance cameras (60 FPS), making them unsuitable for real-time monitoring in practical applications. Furthermore, these algorithms have higher FLOPs, further limiting their deployment in resource-constrained environments.

In contrast, the proposed improved algorithm, while having slightly higher computational complexity compared to YOLOv11, is still lower than YOLOv8 and much lower than RT-DETR-L, SSD300, and YOLOX-s. It maintains an FPS above 60, effectively meeting the real-time requirements for ship monitoring in port channels. Therefore, the proposed algorithm strikes a good balance between detection accuracy and real-time performance while maintaining low hardware dependency, making it highly valuable for practical applications.

Table 3 Performance of different detection methods on mAP@0.5 for each category

Method	mAP@0.5						
	all	tug	Cargo ship	Chemical tanker	Bulk carrier	ferry	Small ship
YOLOv11	90.4%	87.8%	94.5%	95.9%	85.7%	98.0%	80.3%
YOLOv8n [32]	91.0%	85.3%	95.3%	95.0%	86.9%	96.9%	86.7%
YOLOv5n	93.4%	94.4%	94.4%	94.8%	85.4%	97.6%	94.0%
RT-DETR-L [33]	93.6%	85.9%	96.1%	95.7%	86.5%	97.6%	99.5%
YOLOX-s [34]	94.2%	95.5%	94.8%	96.4%	88.5%	97.5%	92.7%
SSD300 [10]	94.3%	96.0%	94.5%	94.4%	89.4%	95.1%	96.1%
Ours	94.8%	96.0%	96.4%	96.5%	90.5%	99.0%	90.2%

Table 4 Performance of different detection methods on mAP@0.5:0.95 for each category

Method	mAP@0.5:0.95						
	all	tug	Cargo ship	Chemical tanker	Bulk carrier	ferry	Small ship
YOLOv11	73.7%	74.4%	81.5%	73.1%	66.8%	84.6%	61.9%
YOLOv8n [32]	75.3%	73.6%	82.1%	75.6%	70.7%	81.5%	68.4%
YOLOv5n	73.8%	77.8%	75.4%	72.4%	67.6%	82.3%	67.5%
RT-DETR-L [33]	77.8%	79.9%	81.6%	75.0%	73.0%	85.0%	72.1%
YOLOX-s [34]	77.5%	80.1%	82.5%	75.5%	71.9%	85.0%	69.7%
SSD300 [10]	76.2%	79.7%	80.8%	73.2%	72.5%	84.0%	75.0%
Ours	78.1%	80.5%	82.5%	75.6%	73.7%	85.4%	71.0%

3.4.2 Specific Detection Performance of Each Category Under Different Detection Methods

To evaluate the detection performance of the proposed improved algorithm across different ship types, comparative experiments were conducted with several mainstream object detection methods, including YOLOv11, YOLOv8n, YOLOv5n, RT-DETR-L, YOLOX-s, and SSD300. The evaluation metrics used were mAP@0.5 and mAP@0.5:0.95, with the results summarized in Tables 3 and 4.

From an overall performance perspective, the proposed algorithm achieves the highest scores in both mAP@0.5 and mAP@0.5:0.95, attaining 94.8% and 78.1%, respectively, outperforming other mainstream methods. This demonstrates its superior detection capability across different IoU thresholds.

For specific ship categories, the improved algorithm also achieves the best or second-best performance on most types in Zhoushan port. For instance, in the most frequently appearing categories-cargo ship, chemical tanker, and bulk carrier, mAP@0.5 exceeds 96%. The ferry category, the second most common type, achieves 99.0%, the highest among all methods. The algorithm maintains a leading position in the mAP@0.5:0.95 metric as well, confirming its robustness and strong recognition ability across diverse ship types in complex port environments.

Notably, for challenging target categories such as tug and small ship, the proposed method also demonstrates effective detection. Although RT-DETR-L slightly outperforms in mAP@0.5 for small ships, the improved algorithm exhibits more stable performance under the stricter mAP@0.5:0.95 metric, indicating superior adaptability and generalization for small targets.

Overall, the YOLOv11-FFCM algorithm outperforms existing mainstream methods in terms of overall accuracy, category balance, and robustness in complex scenarios. It effectively satisfies the dual requirements of detection accuracy and real-time performance for port vessel monitoring, demonstrating strong potential for practical engineering applications.

Table 5 Detection accuracy performance of CBAM at different locations for each category

Category	CBAM-first		CBAM-second		CBAM-third	
	AP@0.5	AP@0.5:0.95	AP@0.5	AP@0.5:0.95	AP@0.5	AP@0.5:0.95
tug	82.5%	64.4%	76.5%	63.9%	96.0%	80.5%
Cargo ship	92.0%	79.3%	94.0%	80.2%	95.4%	82.5%
Chemical tanker	96.4%	73.5%	95.1%	72.3%	96.5%	75.6%
Bulk carrier	87.6%	71.3%	83.3%	64.8%	91.5%	73.7%
ferry	99.1%	85.9%	98.2%	82.8%	99.0%	85.4%
Small ship	87.7%	72.0%	85.1%	65.4%	90.2%	71.0%
all	90.9%	74.4%	88.7%	71.6%	94.8%	78.1%

3.4.3 Comparison of CBAM Detection Performance at Different Locations

To investigate the optimal insertion position of the CBAM attention mechanism within the network, this study embeds it at three different locations (denoted as CBAM-first, CBAM-second, and CBAM-third) and evaluates its impact on category-wise detection performance using a standard dataset. The results are presented in Table 5.

From an overall performance perspective, CBAM-third achieves 94.8% mAP@0.5 and 78.1% mAP@0.5:0.95, significantly outperforming the other configurations. This indicates that embedding the attention module closer to the deeper layers of the network more effectively guides the model to focus on critical target regions, enhances semantic feature representation, and substantially improves detection accuracy.

For individual categories, CBAM-third also demonstrates the best performance, particularly for challenging classes such as small ship and tug, with AP@0.5:0.95 of 71.0% and 80.5%, respectively. These results clearly surpass those of CBAM-first and CBAM-second, highlighting the advantage of deep-layer attention in handling small targets and complex backgrounds.

In contrast, CBAM-first and CBAM-second exhibit slightly lower overall performance, with CBAM-second showing reduced AP@0.5:0.95 for several categories. This may be attributed to the intermediate layer features not yet fully capturing high-level semantic information, limiting the effectiveness of the attention mechanism.

In summary, placing the CBAM module in the deep feature extraction stage (CBAM-third) not only maintains network stability but also enhances the model's focus on key target areas. This configuration delivers stronger detection performance in complex scenarios and for small targets, confirming the significant influence of attention module placement on overall model performance.

3.4.4 Comparison of Detection on the Test Set

To verify the target detection ability of the improved algorithm in complex environments, this paper tests the original YOLOv11 algorithm and the improved algorithm in the test set (the original algorithm is on the left and the improved algorithm is on the right).

As shown in Fig. 13, in the complex background detection task, the original algorithm mistakenly identified the bulk carrier as a ferry, while the improved algorithm effectively avoided this misjudgment, significantly enhancing the model's robustness. Under foggy conditions, the original algorithm failed to detect the transport ship on the far right of the image. In contrast, the improved algorithm successfully identified the target, demonstrating its enhanced detection capabilities in low-visibility environments. Furthermore, under occlusion conditions, the original algorithm failed to detect the bulk carrier, while the improved algorithm successfully identified the target, confirming its ability to detect objects in the presence of occlusion.

Fig. 13 Comparison results on the test set

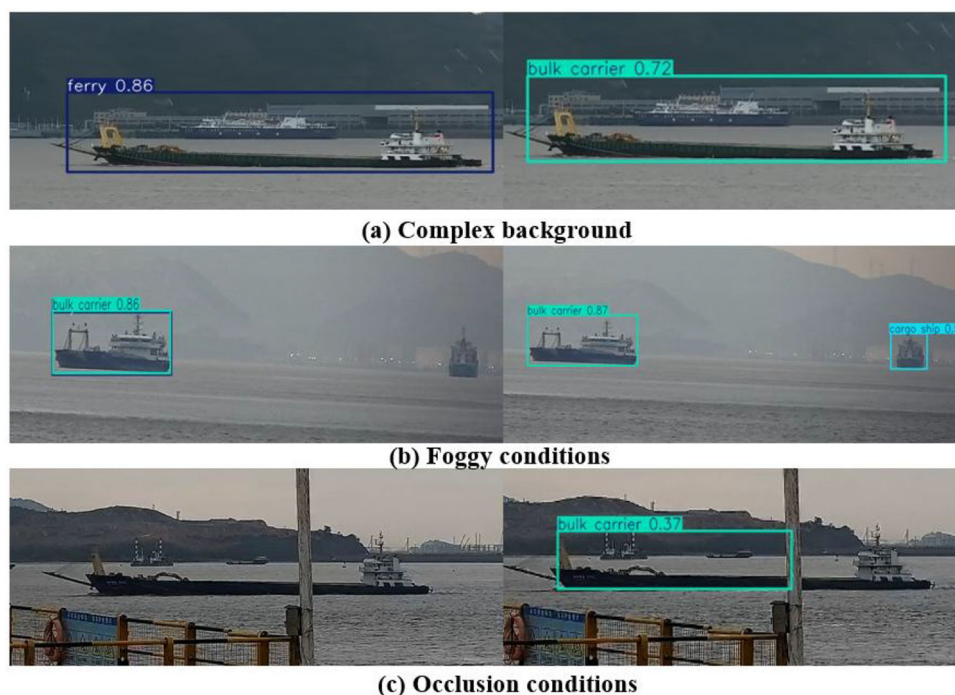


Table 6 Performance comparison of different methods with improvement strategies

Method	Improvement Strategy			P	R	mAP @0.5	mAP @0.5:0.95	FPS /s ⁻¹
	FocalModulation	C3k2-FasterNetBlock	CBAM					
YOLOv11				92.6%	80.0%	90.4%	73.7%	97
A	+			91.0%	86.3%	91.5%	74.6%	95
B	+	+		91.5%	87.1%	92.5%	75.4%	94
Ours	+	+	+	93.5%	87.8%	94.8%	78.1%	90

3.5 Ablation Experiments

To assess the contributions of different improvement strategies in the YOLOv11-FFCM algorithm, ablation experiments were conducted on FocalModulation, C3k2-FasterNetBlock, and the CBAM attention mechanism. The experiments used the HarbourShip dataset, and five evaluation metrics were employed: Precision, Recall, mAP@0.5, mAP@0.5:0.95, and FPS. Training hyperparameters were kept consistent, as detailed in Table 1, and the experimental results are summarized in Table 6.

As shown in Table 6 and Fig. 14, the original YOLOv11 model, without any enhancement modules, achieves an mAP@0.5 of 90.4% and an mAP@0.5:0.95 of 73.7%, with a recall of 80.0% and an FPS of 97. This indicates strong real-time performance, but still leaves room for improvement in detection accuracy under complex environments.

After introducing the Focal Modulation module (Strategy A), the model's mAP@0.5 increases to 91.5%, and mAP@0.5:0.95 also rises slightly to 74.6%. Moreover, the recall improves significantly to 86.3%, suggesting that Focal Modulation effectively enhances the model's contextual awareness, thereby improving detection performance.

Replacing the backbone with the FasterNetBlock structure (Strategy B) further boosts the mAP@0.5 to 92.5% and mAP@0.5:0.95 to 75.4%, with the recall reaching 87.1%. Meanwhile, the FPS only drops by one frame, demonstrating that this module maintains high accuracy while controlling computational complexity and improving feature extraction efficiency.

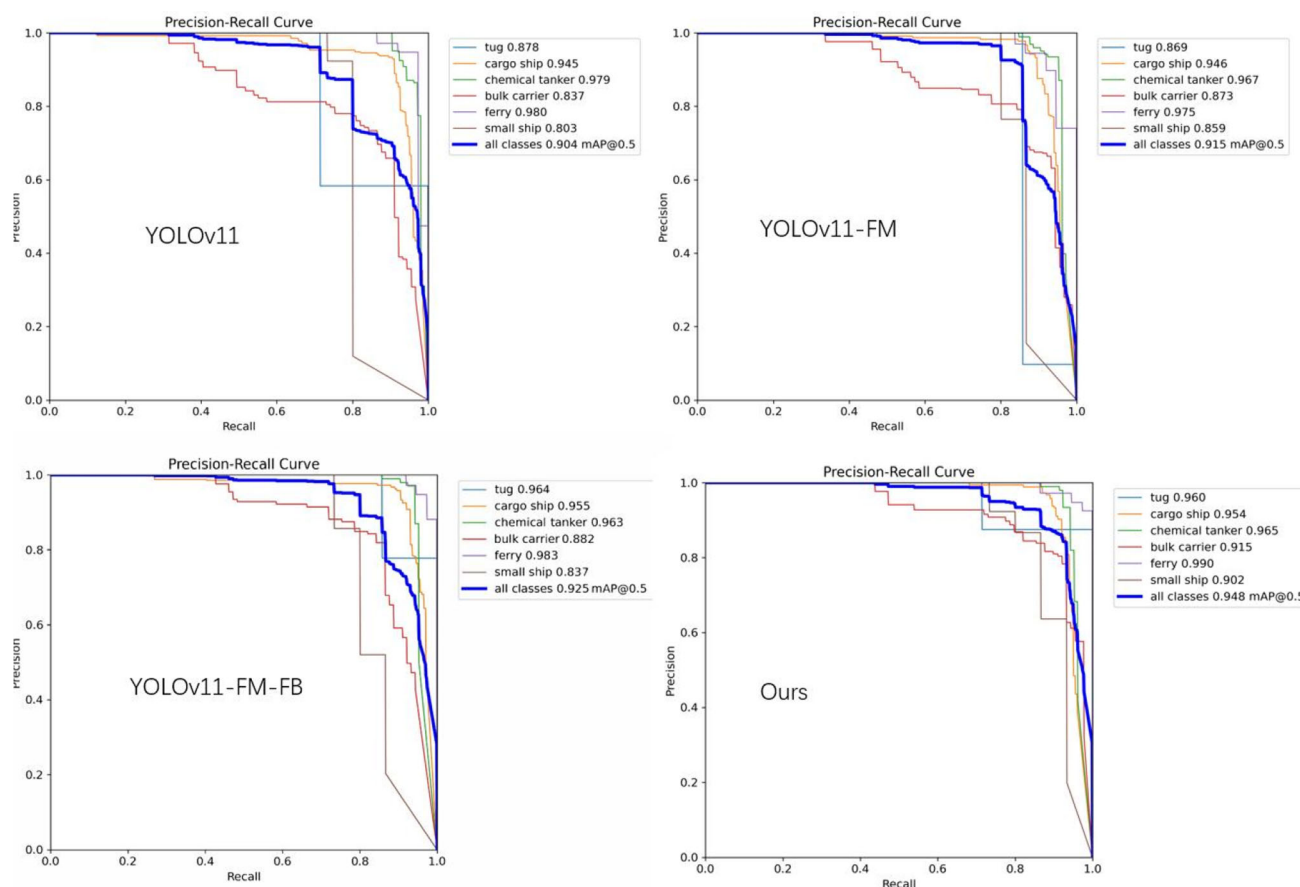


Fig. 14 Dataset example

To address the slight drop in precision caused by the previous modules, the CBAM attention mechanism is finally integrated. This results in a further increase in performance, with mAP@0.5 and mAP@0.5:0.95 reaching 94.8% and 78.1%, respectively. Precision and recall improve to 93.5% and 87.8%. Although FPS decreases slightly to 90, it still exceeds the standard requirement of 60 FPS for real-time monitoring. CBAM enhances the model's focus on critical target regions through both channel and spatial attention, further improving detection accuracy in complex scenarios.

In summary, the proposed improvement strategies significantly enhance detection accuracy while maintaining real-time performance. The final YOLOv11-FFCM model achieves a balanced trade-off between accuracy, computational efficiency, and real-time capability, demonstrating strong practical value and potential for widespread application.

3.6 Generalization Experiment

To validate the generalization ability of the proposed YOLOv11-FFCM algorithm on unseen datasets, we conducted generalization experiments on the public ship object detection dataset Unreal Vessels Detection v1, and compared its performance with several mainstream detection algorithms, including YOLOv8, YOLOX, SSD, and RT-DETR. The performance comparison results on this dataset are presented in Table 7. As shown in Fig. 15, the Unreal Vessels-Detection-v1 dataset contains a total of 10,147 images, with several representative examples provided.

From the results, YOLOv11-FFCM demonstrates outstanding performance across all evaluation metrics. Specifically, it achieves a Precision of 98.8% and a Recall of 74.5%. The mAP@0.5 and mAP@0.5:0.95 reach 98.8% and 85.6%, respectively, representing the highest values among all the compared methods. These findings pro-

Table 7 Generalization performance comparison of different methods on the Unreal-Vessels-Detection-v1 dataset

Method	P	R	mAP@0.5	mAP@0.5:0.95	FLOPs/G	FPS
YOLOv5n	96.3%	73.5%	86.2%	78.8%	7.1	96
YOLOv8n [32]	92.1%	72.8%	94.5%	82.3%	8.1	132
YOLOv11	98.0%	74.3%	96.6%	83.2%	6.3	97
RTDETR-L [33]	97.4%	73.8%	94.4%	81.8%	103.5	24
SSD300 [10]	97.7%	74.0%	94.7%	82.8%	61.3	35
YOLOX-s [34]	98.6%	74.3%	97.8%	84.1%	26.8	50
Ours	98.8%	74.5%	98.8%	85.6%	7.5	90

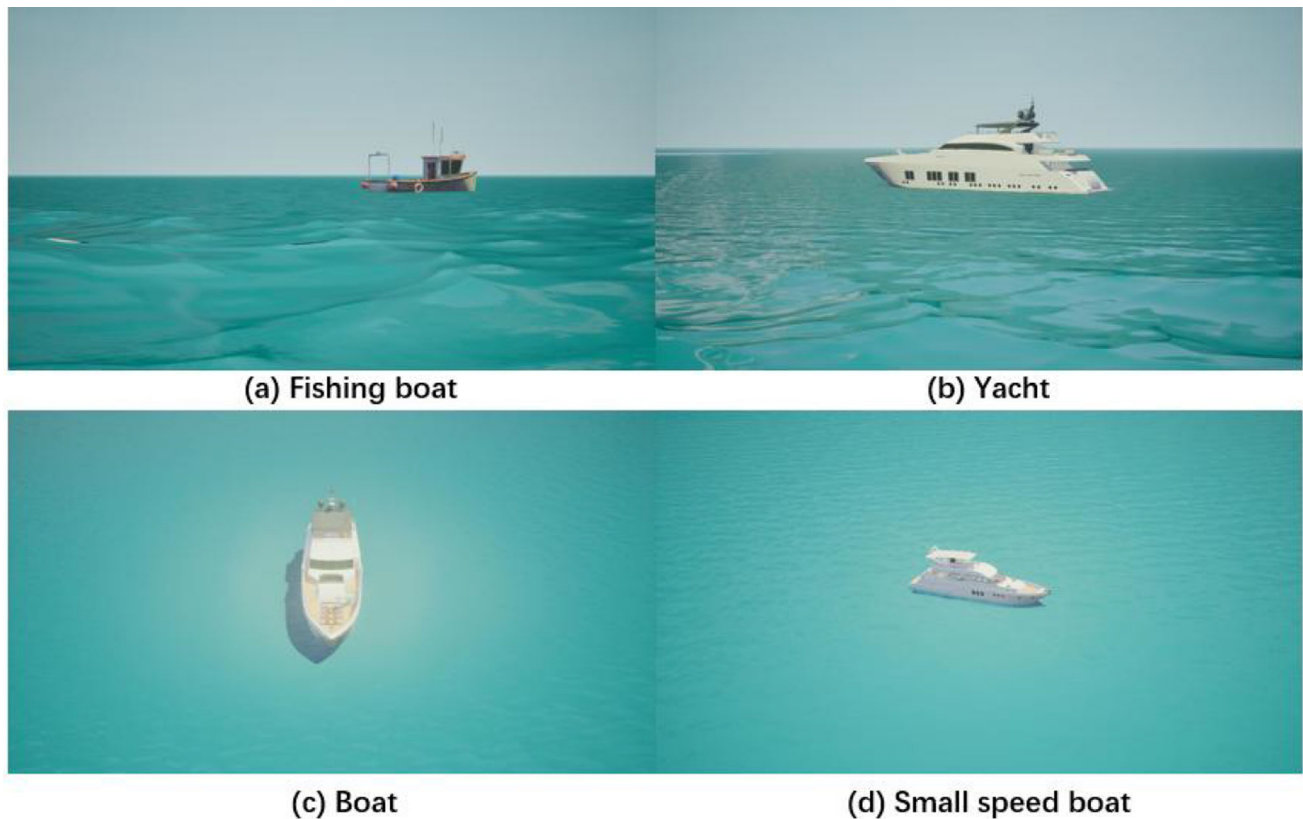


Fig. 15 Dataset example

vide strong evidence of the model's superior detection capabilities and robust generalization ability in complex scenarios.

Although RT-DETR-L, SSD300, and YOLOX-s also demonstrate relatively high detection accuracy, they are associated with significantly greater computational complexity. For example, YOLOX-s achieves mAP@0.5 and mAP@0.5:0.95 of 97.8% and 84.1%, respectively, while SSD300 reaches 82.8% in mAP@0.5:0.95. However, the corresponding FLOPs of these models are 103.5G, 61.3G, and 26.8G, which impose substantial demands on hardware resources. In addition, their inference speeds are relatively low, with FPS values of 24, 35, and 50, respectively, limiting their suitability for high-frame-rate real-time monitoring tasks.

Although YOLOv8n achieves the highest inference speed at 132 FPS, its detection performance is inferior to the proposed method, achieving only 94.5% and 82.3% in mAP@0.5 and mAP@0.5:0.95, respectively.

Overall, YOLOv11-FFCM delivers superior detection accuracy and robustness while maintaining a low computational burden (7.5 GFLOPs) and a high inference speed (90 FPS). These characteristics highlight its strong

potential for practical deployment, particularly in vessel monitoring scenarios within ports and waterways that require both real-time performance and strong generalization capabilities.

4 Conclusion

This study aims to achieve real-time ship detection in the Zhoushan port and waterway monitoring system, particularly for high-precision identification of ships of varying sizes and orientations under complex backgrounds and low-visibility conditions. To address the limitations of existing detection methods that struggle to balance detection accuracy, hardware adaptability, and real-time performance, this paper proposes an optimized detection algorithm based on an improved YOLOv11 framework, called YOLOv11-FFCM. This algorithm integrates the Focal Modulation module, FasterNetBlock structure, and CBAM attention mechanism, optimizing the neural network architecture to enhance feature extraction and information interaction, thereby improving detection performance.

Experimental results show that, compared with the original YOLOv11, the proposed method improves Precision by 2.9%, Recall by 7.8%, mAP@0.5 by 4.4%, and mAP@0.5:0.95 by 4.4% on the Harbour Ship dataset. It demonstrates stronger robustness in challenging scenarios, such as low visibility, complex backgrounds, and long-range small target detection.

Furthermore, when compared with mainstream detection methods, such as YOLOv5n, YOLOv8n, YOLOX, SSD, and RTDETR, the proposed method achieves superior detection accuracy on both the public dataset Unreal Vessels detection.v1 and the custom Harbour Ship dataset, demonstrating strong generalization capability. At the same time, it achieves a better balance between computational complexity and real-time performance, making it more suitable for deployment on edge devices in Zhoushan port and waterway monitoring systems.

Future work will focus on further exploring lightweight models to enhance computational efficiency, reduce deployment costs, and deploy the proposed ship detection model on high-performance edge computing devices to facilitate practical applications.

Author Contributions Conceptualization: Q.X. and B.X.; methodology: Q.X. and B.X.; formal analysis: Q.X., B.X., and C.P.; data curation: Q.X., B.X., and Q.L.; schematic design: Q.X.; writing-original draft preparation: Q.X., B.X., and F.B.; writing-review and editing: Q.X. and B.X.; supervision: B.X., C.P., and Q.L.; project administration: B.X. and F.B. All authors have read and agreed to the published version of the manuscript.

Funding This work was supported by Shanghai Rising-Star Program (Grant No. 240B2707000)

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethical Approval This research does not involve human participants, animals, or sensitive data, and thus does not require ethical approval.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Rekavandi, A.M., Xu, L., Boussaid, F., Seghouane, A.-K., Hoefs, S., Bennamoun, M.: A guide to image-and video-based small object detection using deep learning: Case study of maritime surveillance. *IEEE Transactions on Intelligent Transportation Systems* (2025)
2. Ribeiro, C.V., Paes, A., Oliveira, D.: Ais-based maritime anomaly traffic detection: A review. *Expert Syst. Appl.* **231**, 120561 (2023)
3. Bao, X., Javanbakhti, S., Zinger, S., Wijnhoven, R.G.J., With, P.H.N.: Context modeling combined with motion analysis for moving ship detection in port surveillance. *Journal of Electronic Imaging* (2013)
4. Zhi, L., Chang-wen, Q., Qiang, Z., Chen, L., Shujuan, P., Jianwei, L.: Ship detection in harbor area in sar images based on constructing an accurate sea-clutter model. 2017 2ND INTERNATIONAL CONFERENCE ON IMAGE, VISION AND ... (2017)
5. Er, M.J., Zhang, Y., Chen, J., Gao, W.: Ship detection with deep learning: A survey. *Artif. Intell. Rev.* **56**(10), 11825–11865 (2023)
6. Wang, X., Liu, J., Liu, X., Liu, Z., Khalaf, O.I., Ji, J., Ouyang, Q.: Ship feature recognition methods for deep learning in complex marine environments. *Complex & Intelligent Systems* **8**(5), 3881–3897 (2022)
7. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016)
8. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **39**(6), 1137–1149 (2016)
9. Han, X., Chang, J., Wang, K.: You only look once: Unified, real-time object detection. *Procedia Computer Science* **183**(1), 61–72 (2021)
10. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: Ssd: Single shot multibox detector. In: *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14*, pp. 21–37 (2016). Springer
11. Olorunshola, O.E., Irhebhude, M.E., Ewwiekpaefe, A.E.: A comparative study of yolov5 and yolov7 object detection algorithms. *Journal of Computing and Social Informatics* **2**(1), 1–12 (2023)
12. Zhao, H., Zhang, W., Sun, H., Xue, B.: Embedded deep learning for ship detection and recognition. *FUTURE INTERNET* (2019)
13. Liu, D., Shi, G.: Ship collision risk assessment based on collision detection algorithm. *IEEE ACCESS* (2020)
14. Wenbin, G., Zhangsong, S., Chengxu, F., Zhonghong, W.: Port ship detection based on visual saliency model and center dark channel prior. 2020 INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE ... (2020)
15. Hu, X., Liu, S., Wu, Z., Chen, Z.: Research on a port ship target detection method based on lightweight multi-scale convolutional network (2022)
16. Shi, J., Liu, Z.: Track pairs collision detection with applications to ship collision risk assessment. *JOURNAL OF MARINE SCIENCE AND ENGINEERING* (2022)
17. Kim, T.-h., Park, Y., Jang, S., Yang, C.-S.: Ship detection and classification in cas500-1 images based on yolo model. *IGARSS 2023 - 2023 IEEE INTERNATIONAL GEOSCIENCE AND REMOTE ...* (2023)
18. Bakirci, M., Bayraktar, I.: Improving coastal and port management in smart cities with uavs and deep learning. 2024 MEDITERRANEAN SMART CITIES CONFERENCE (MSCC) (2024)
19. Li, G., Zhang, X., Shu, Y., Wang, C., Guo, W., Wang, J.: Ship anomalous behavior detection in port waterways based on text similarity and kernel density estimation. *JOURNAL OF MARINE SCIENCE AND ENGINEERING* (2024)
20. Qiu, M., Huang, L., Tang, B.-H.: Asff-yolov5: Multielement detection method for road traffic in uav images based on multiscale feature fusion. *Remote Sensing* **14**(14), 3498 (2022)
21. Misra, I., Girdhar, R., Joulin, A.: An end-to-end transformer model for 3d object detection. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2906–2917 (2021)
22. Jin, X., Xie, Y., Wei, X.-S., Zhao, B.-R., Chen, Z.-M., Tan, X.: Delving deep into spatial pooling for squeeze-and-excitation networks. *Pattern Recogn.* **121**, 108159 (2022)
23. Chen, H., Wang, Y., Guo, J., Tao, D.: Vanillanet: the power of minimalism in deep learning. *Adv. Neural. Inf. Process. Syst.* **36**, 7050–7064 (2023)
24. Wang, A., Chen, H., Lin, Z., Han, J., Ding, G.: Repvit: Revisiting mobile cnn from vit perspective. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15909–15920 (2024)
25. Xu, G., Li, J., Gao, G., Lu, H., Yang, J., Yue, D.: Lightweight real-time semantic segmentation network with efficient transformer and cnn. *IEEE Trans. Intell. Transp. Syst.* **24**(12), 15897–15906 (2023)
26. He, K., Zhang, X., Ren, S., Sun, J.: Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* **37**(9), 1904–1916 (2015)
27. Yang, J., Li, C., Dai, X., Gao, J.: Focal modulation networks. *Adv. Neural. Inf. Process. Syst.* **35**, 4203–4217 (2022)
28. Luo, Y., Ou, Z., Wan, T., Guo, J.-M.: Fastnet: Fast high-resolution network for human pose estimation. *Image Vis. Comput.* **119**, 104390 (2022)

29. Targ, S., Almeida, D., Lyman, K.: Resnet in resnet: Generalizing residual architectures. arXiv preprint [arXiv:1603.08029](https://arxiv.org/abs/1603.08029) (2016)
30. Zhao, H., Zhang, Y., Liu, S., Shi, J., Loy, C.C., Lin, D., Jia, J.: Pscanet: Point-wise spatial attention network for scene parsing. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 267–283 (2018)
31. Woo, S., Park, J., Lee, J.-Y., Kweon, I.S.: Cbam: Convolutional block attention module. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 3–19 (2018)
32. Varghese, R., Sambath, M.: Yolov8: A novel object detection algorithm with enhanced performance and robustness. In: 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS), pp. 1–6 (2024). IEEE
33. Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: Detrs beat yolos on real-time object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 16965–16974 (2024)
34. Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: Yolox: Exceeding yolo series in 2021. arXiv preprint [arXiv:2107.08430](https://arxiv.org/abs/2107.08430) (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Boweng Xing¹ · Qingsong Xing¹ · Chengwu Pan² · Qibo Le² · Fujia Bi³

✉ Boweng Xing
bwxing@shou.edu.cn

Qingsong Xing
xgreenpine@163.com

Chengwu Pan
nbpcw@163.com

Qibo Le
leqibo@126.com

Fujia Bi
mingzhou2020@163.com

¹ College of Engineering Science and Technology, Shanghai Ocean University, Shanghai 201306, China

² Ningbo Communication Center, Ningbo 315800, China

³ Beijing Mingzhou Technology Co.Ltd., Beijing 316299, China